

Concepteur et développeur d'application

Rapport All Scans



I. Liste des compétences mises en œuvre.....	4
Matrice de couverture des compétences CDA.....	4
II. Cahier des charges / expression des besoins.....	5
1. Genèse et Problématique.....	5
2. Objectifs du projet et Solutions logicielles.....	5
3. Identification des utilisateurs cibles.....	6
4. Liste exhaustive des fonctionnalités.....	7
III. Présentation de l'entreprise.....	8
IV. Gestion de projet.....	9
1. Organisation et suivi des tâches.....	9
2. Stratégie de versioning.....	9
3. Intégration Continue et Qualité Logicielle.....	10
V. Spécifications fonctionnelles.....	12
1. Conception des interfaces.....	12
2. Modélisation UML : Diagramme de Cas d'Utilisation.....	14
VI. Architecture logicielle.....	16
1. Architecture globale de l'application.....	16
2. Modélisation : Du Conceptuel au Physique.....	17
3. Structure des Documents et Stratégies d'Optimisation.....	17
VII. Spécifications techniques.....	21
1. Organisation du code et conventions de nommage.....	21
2. Mise en conformité RGPD.....	21
3. Accessibilité.....	22
4. Gestion des erreurs et résilience.....	22
5. Intégration et conformité de l'API Scryfall.....	23
VIII. Réalisations.....	23
1. Développement Backend.....	23
Structuration des routes API et cohérence des données.....	24
Interfaçage avec MongoDB et opérations asynchrones complexes.....	24
Logique métier et flexibilité des formats.....	26
2. Développement Frontend.....	29
Architecture des composants.....	29
Gestion de l'état.....	30
Appels à l'API FastAPI et Optimisations de performance.....	31
3. Déploiement et Conteneurisation.....	35
Le principe de l'environnement isochrone.....	35
Image Docker dédiée au Backend.....	35
Orchestration avec Docker Compose.....	37
IX. Sécurité de l'application.....	38

1. Authentification et Autorisation.....	38
Le choix des jetons opaques.....	38
Sécurisation du transport via les Cookies.....	38
Authentification Multifacteur.....	39
2. Sécurisation des données en base.....	40
Le choix de l'algorithme Argon2.....	40
Implémentation et évolution.....	40
3. Protection contre les vulnérabilités courantes.....	41
Prévention des Injections NoSQL.....	41
Filtrage des requêtes externes.....	42
X. Plan de tests et jeu d'essai.....	42
1. Présentation de la stratégie de test.....	42
2. Tests unitaires et d'intégration.....	43
Robustesse de la logique métier (Test Unitaire).....	43
Validation des flux complexes (Test d'Intégration).....	43
3. Tests de performance et de charge.....	43
Scalabilité de la base de données (MongoDB).....	44
Performances CPU et appels réseau.....	44
4. Jeu d'essai représentatif.....	44
XI. Veille sécurité.....	45
1. Suivi de l'écosystème de développement.....	45
2. Sécurité logicielle et prévention des failles (OWASP).....	46
Conclusion et axes d'amélioration.....	47
1. Bilan personnel sur le travail en autonomie.....	47
2. Perspectives d'évolution de l'application.....	47
XII. Annexes.....	48

I. Liste des compétences mises en œuvre.

Matrice de couverture des compétences CDA

La réalisation de cette application a permis de mobiliser et de valider les compétences du référentiel Concepteur Développeur d'Applications, réparties selon les trois activités types :

Bloc de compétences	Compétences mises en œuvre dans le projet
Développer une application sécurisée (Frontend)	- Maquettage des interfaces (Wireframes des vues de recherche et de création de decks). - Développement d'interfaces utilisateur dynamiques avec React (composants isolés, gestion d'état local). - Consommation de l'API externe Scryfall et de l'API interne. - Sécurisation des entrées utilisateur côté client.
Concevoir et développer une application sécurisée organisée en couches (Backend)	- Modélisation des données adaptées aux contraintes de flexibilité de l'API externe (utilisation d'une base de données orientée document MongoDB). - Développement d'une API RESTful asynchrone avec FastAPI (Python). - Implémentation des règles métiers (validation des formats de jeu, moteur de tags, logique d'allocation physique). - Sécurisation des accès (hachage des mots de passe, gestion des cookies de session HttpOnly).
Préparer le déploiement d'une	- Conteneurisation de l'environnement de

application sécurisée (DevOps)	développement et de production via Docker (rédaction des Dockerfile et docker-compose.yml). - Conception et exécution d'un plan de test exhaustif (tests unitaires et d'intégration automatisés). - Mise en place d'une chaîne d'Intégration Continue (CI) avec GitHub Actions pour l'exécution systématique des tests backend.
---------------------------------------	--

II. Cahier des charges / expression des besoins

1. Genèse et Problématique

Le marché des jeux de cartes à collectionner (TCG), et plus particulièrement celui de *Magic : The Gathering*, implique pour les joueurs réguliers la gestion d'un volume de données massif et en constante évolution. Actuellement, l'écosystème propose plusieurs solutions logicielles professionnelles dédiées à la gestion de collection (telles que ManaBox, Moxfield ou MythicTools). Bien que ces outils permettent un tri et un référencement numérique efficaces, ils présentent une limite structurelle majeure : l'incapacité à superviser la réalité physique de la collection lors de la phase de "Deck Building" (construction de paquets de cartes).

La problématique principale réside dans la gestion des conflits d'allocation physique. Dans les applications existantes, un utilisateur possédant un unique exemplaire d'une carte spécifique peut l'inclure virtuellement dans une multitude de decks différents. Cependant, dans la réalité physique du jeu, cette carte ne peut être présente que dans un seul deck "construit" à un instant T. Il manque un pont logique entre le référencement numérique et la contrainte de disponibilité matérielle.

2. Objectifs du projet et Solutions logicielles

Pour répondre à cette problématique, ce projet d'application web propose un outil de gestion et de deck-building axé sur la réalité matérielle de l'utilisateur. Le développement a été mené en totale autonomie afin de couvrir l'ensemble du cycle de vie du logiciel.

Les objectifs fonctionnels majeurs se déclinent ainsi :

- **Gestion d'allocation stricte** : Implémentation d'un statut de "deck construit". Lorsqu'une carte de la collection physique est assignée à un deck possédant ce statut, elle est verrouillée logiciellement et devient indisponible pour la construction d'un autre deck physique, garantissant ainsi la cohérence de l'inventaire en temps réel.
- **Moteur de recherche et référencement** : Intégration de l'API REST Scryfall, référence absolue du domaine, permettant d'interroger une base de données exhaustive et d'appliquer des filtres de recherche personnalisés et complexes.
- **Outils d'optimisation analytique** : Intégration de modules de validation des règles de construction (respect des formats de jeu) et de génération de graphiques statistiques visant à optimiser la "base de mana" (ressources du jeu) des decks créés.
- **Interopérabilité** : Développement de fonctionnalités d'importation et d'exportation de collections en masse.

Note de dimensionnement du projet : La conception initiale inclut la modélisation d'une station robotisée (via un microcontrôleur ESP32 et ESP32cam) destinée à automatiser le tri physique des cartes. Afin de respecter les délais impartis et le périmètre d'évaluation du titre CDA, ce module matériel a été écarté du développement logiciel. Les plans de conception et la nomenclature matérielle, démontrant l'étude de faisabilité, sont néanmoins fournis en annexes.

3. Identification des utilisateurs cibles

Le développement de cette application est orienté pour répondre aux besoins spécifiques de deux profils d'utilisateurs distincts, bien que souvent complémentaires dans l'écosystème des jeux de cartes à collectionner :

- **Le Joueur (Régulier à Compétitif) :** Cet utilisateur utilise l'application comme un outil d'optimisation stratégique (Deck-building). Ses besoins principaux sont la vérification de la légalité de ses decks selon les formats de tournoi (ex: Standard, Commander), l'analyse statistique de sa courbe de mana (ressources), et surtout, la garantie que ses decks virtuels correspondent à la réalité de sa disponibilité matérielle (gestion stricte des allocations physiques).
- **Le Collectionneur :** Cet utilisateur privilégie la dimension d'inventaire. Il a besoin d'outils d'importation et d'exportation massives, d'une interface de recherche exhaustive (interfaçage avec Scryfall) pour identifier des éditions spécifiques, et d'un système de listes de souhaits (wishlists) pour préparer de futurs achats ou échanges.

4. Liste exhaustive des fonctionnalités

Afin de répondre aux objectifs fixés, l'application intègre les modules fonctionnels suivants, validant les compétences de développement front-end et back-end :

- **Module d'Authentification et de Sécurité :**
 - Inscription et connexion des utilisateurs.
 - Gestion sécurisée des sessions via cookies **HttpOnly** pour prévenir les attaques XSS.
 - Hachage des mots de passe en base de données.
 - Authentification renforcé avec microsoft authenticator
- **Module de Recherche et de Consultation (Interfaçage API Scryfall) :**
 - Recherche textuelle simple (par nom de carte).
 - Recherche multicritères avancée (par couleur exacte, couleur relative, coût de mana, type, sous-type, texte de règles, édition).
 - Affichage détaillé d'une carte (visuel, prix, légalité dans les formats).
 - Redirection vers des sites vendeurs
 - Saisie de Tags personnalisé (une liste de règles définit par l'utilisateur)

-
- **Module de Gestion de Collection (Inventaire) :**
 - Ajout, modification et suppression de cartes dans la collection physique.
 - Importation et exportation de listes de cartes (formats texte).
 - Suivi en temps réel du volume global de la collection.
 - **Module de Deck Building (Cœur de l'application) :**
 - Création, édition et suppression de decks.
 - *Règle métier critique* : Blocage de l'allocation d'une carte physique à un deck "construit" si l'exemplaire est déjà verrouillé dans un autre deck de même statut.
 - Génération de statistiques visuelles (graphiques de répartition des couleurs et de la courbe de mana).
 - Validation algorithmique de la légalité du deck par rapport aux règles du format choisi.

III. Présentation de l'entreprise

Bien que le référentiel de certification prévoie la possibilité de réaliser le projet d'application au sein d'une entreprise d'accueil, le développement de la solution "All Scans" a été mené de manière totalement indépendante et autonome. Il s'inscrit dans le cadre de la validation du titre professionnel de Concepteur Développeur d'Applications (CDA) préparé au sein de l'École Hexagone.

Ce choix d'un projet personnel répondant à une problématique bien réelle (la gestion d'inventaire physique avec allocation stricte pour les joueurs de TCG) m'a permis d'assumer l'intégralité des rôles inhérents au cycle de vie d'un logiciel.

N'étant pas contraint par l'infrastructure préexistante d'une entreprise, j'ai pu agir à la fois comme :

- **Product Owner** : Conception du cahier des charges, identification des cibles et priorisation des fonctionnalités.
- **Architecte Logiciel** : Sélection technologique stratégique (FastAPI pour l'asynchronisme, MongoDB pour la flexibilité des documents NoSQL).

- **Développeur Full-Stack** : Implémentation complète de l'interface utilisateur (React) et de la logique métier sécurisée (Python).
- **Ingénieur DevOps** : Mise en place de l'intégration continue (GitHub Actions) et de la conteneurisation (Docker).

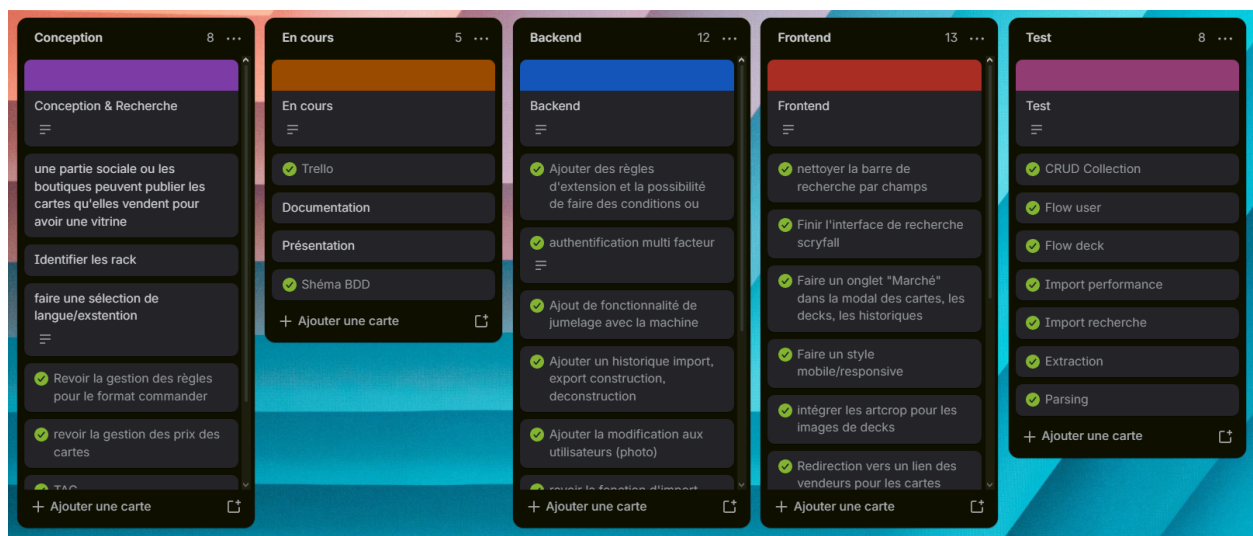
Ce cadre de travail en autonomie totale garantit la mobilisation directe et la pleine maîtrise de l'ensemble des compétences évaluées par le présent titre professionnel.

IV. Gestion de projet

1. Organisation et suivi des tâches

Le projet ayant été réalisé en totale autonomie, la méthodologie adoptée s'est inspirée des principes agiles (Kanban) afin de maintenir un flux de développement continu et structuré. L'organisation du travail a été centralisée sur l'outil **Trello**.

Cette approche a permis de découper le cahier des charges en tâches atomiques et de visualiser l'avancement global du projet. Le tableau a été structuré de manière classique pour isoler les phases de conception, de développement backend, de développement frontend, et de tests.



2. Stratégie de versioning

La gestion du code source a été assurée via Git et hébergée sur GitHub. La topologie de l'équipe (développeur unique) a conduit à l'adoption d'une stratégie de versioning simplifiée et directe, s'apparentant au **Trunk-Based Development**.

Le développement s'est effectué exclusivement sur une branche principale unique. Les validations de code (commits) et les envois sur le dépôt distant (pushes) ont été réalisés de manière asynchrone, généralement après la complétion d'un bloc fonctionnel entier (plusieurs heures ou jours de développement). Ce choix d'un historique de commits orienté "jalons" (milestones) plutôt que "micro-tâches" a permis de conserver une traçabilité claire des grandes étapes de l'implémentation logicielle, tout en évitant la surcharge administrative liée à la gestion de multiples branches de fonctionnalités (feature branches).

3. Intégration Continue et Qualité Logicielle

Bien que le cycle de push soit espacé, la fiabilité du code est garantie par une chaîne d'Intégration Continue (CI) rigoureuse, configurée via **GitHub Actions**.

Un workflow automatisé, défini dans le fichier `test-backend.yml`, se déclenche systématiquement à chaque nouveau push sur le dépôt distant. Ce pipeline a pour responsabilité d'exécuter l'ensemble de la suite de tests contenue dans le répertoire `test/` du projet. Cette architecture garantit qu'aucune régression fonctionnelle n'est introduite sur la branche principale lors de l'ajout de nouvelles fonctionnalités (comme l'optimisation des règles métier ou l'interfaçage avec l'API Scryfall), assurant ainsi un niveau de qualité constant tout au long du cycle de vie du logiciel.

À titre d'exemple, voici un extrait du workflow GitHub Actions implémenté (`.github/workflows/test-backend.yml`). Ce script illustre l'automatisation du processus de vérification : à chaque nouveau "push", un environnement virtuel Ubuntu est instancié, les dépendances Python sont installées, et la suite de tests `pytest` est exécutée.

```
name: Backend CI
# Déclencheurs : Quand est-ce qu'on lance les tests ?
on:
  push:
    branches: [ "main", "master" ] # À chaque push sur la branche
```

```
principale
  pull_request:
    branches: [ "main", "master" ] # À chaque Pull Request

jobs:
  test-backend:
    runs-on: ubuntu-latest

    # On lance un MongoDB temporaire pour les tests d'intégration
    services:
      mongo:
        image: mongo:4.4
        ports:
          - 27017:27017
        options: >-
          --health-cmd mongo
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      # GitHub récupère le code
      - name: Checkout code
        uses: actions/checkout@v3

      # Installation de Python
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      # Installation des librairies
      - name: Install dependencies
        working-directory: ./backend
        run: |
          python -m pip install --upgrade pip
```

```
    pip install -r requirements.txt
    # On installe explicitement pytest si jamais il manque au
requirements.txt
    pip install pytest pytest-asyncio httpx pyotp

# Tests Unitaires
- name: Run Unit Tests
  working-directory: ./backend
  run: python -m pytest tests/unit

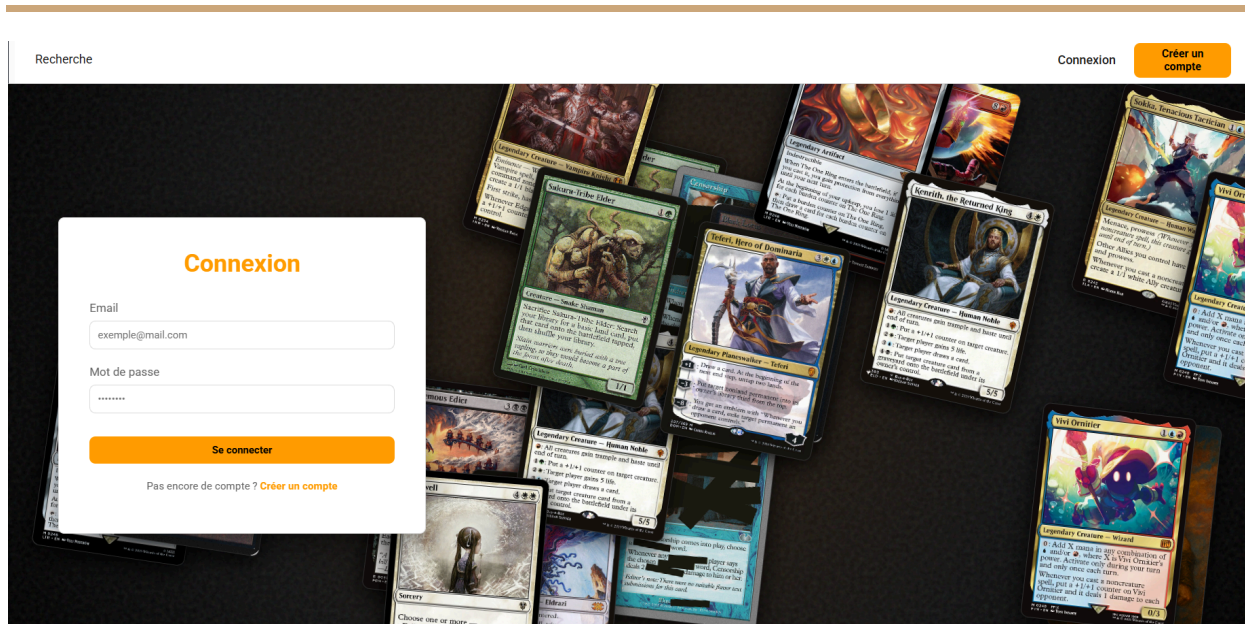
# Tests d'Intégration
- name: Run Integration Tests
  working-directory: ./backend
  env:
    MONGO_DB_NAME: All_scans_TEST # Variable d'env pour isoler la
BDD
  run: python -m pytest tests/integration
```

V. Spécifications fonctionnelles

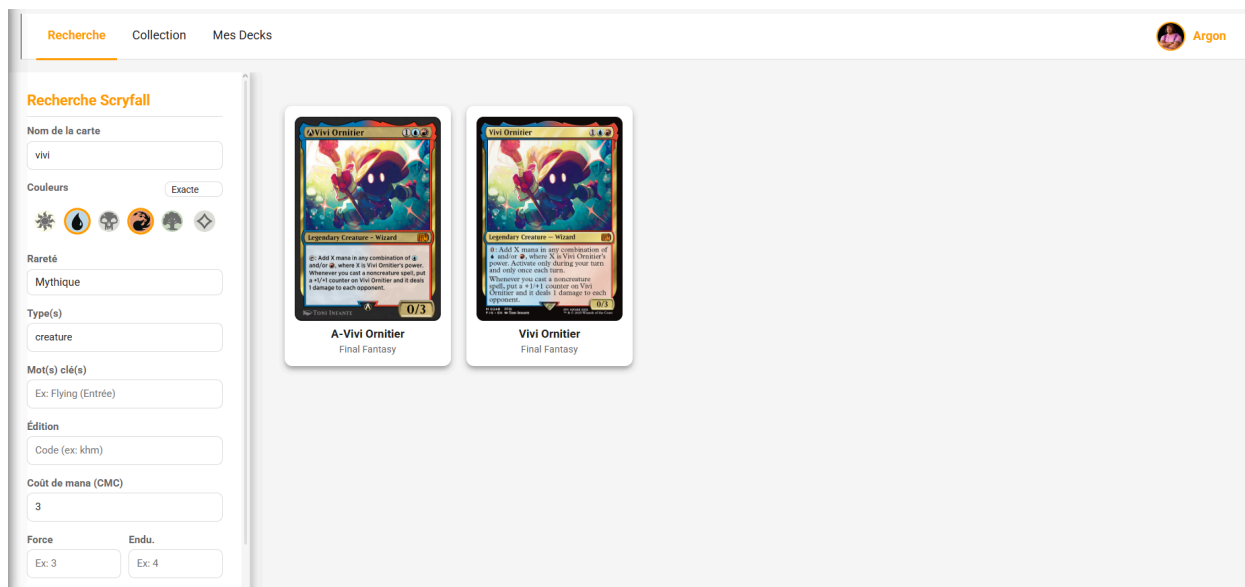
1. Conception des interfaces

L'Interface Homme-Machine (IHM) a été pensée pour être réactive et offrir une expérience utilisateur (UX) fluide, capable de gérer l'affichage de nombreuses données visuelles.

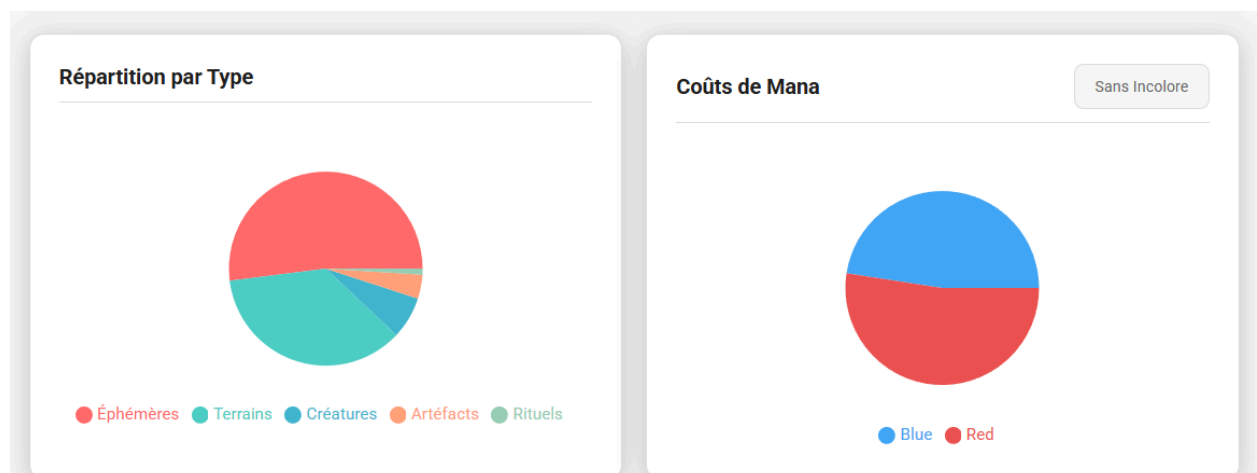
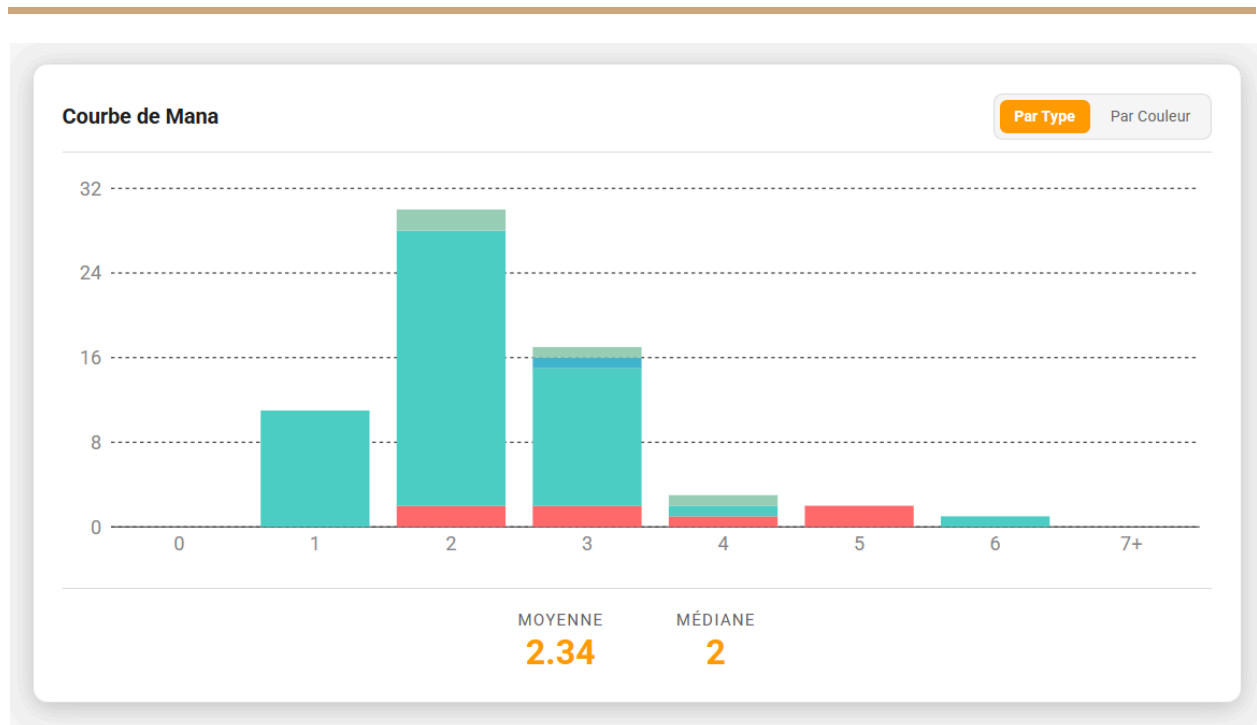
- **Écran d'Authentification ([LoginPage.js](#))** : Interface épurée centralisant l'accès à l'application, avec gestion des retours d'erreurs (identifiants incorrects).



- **Interface de Recherche (ScryfallSearch.js)** : Composant central permettant de requêter l'API Scryfall. L'interface propose des filtres dynamiques permettant de réduire la granularité des résultats.



- **Espace de Gestion de Deck (DeckDetails.js)** : Cette vue constitue le tableau de bord stratégique du joueur. Elle affiche la liste des cartes allouées, le statut du deck (en construction ou construit/verrouillé) et les composants graphiques représentant les statistiques du deck.



2. Modélisation UML : Diagramme de Cas d'Utilisation

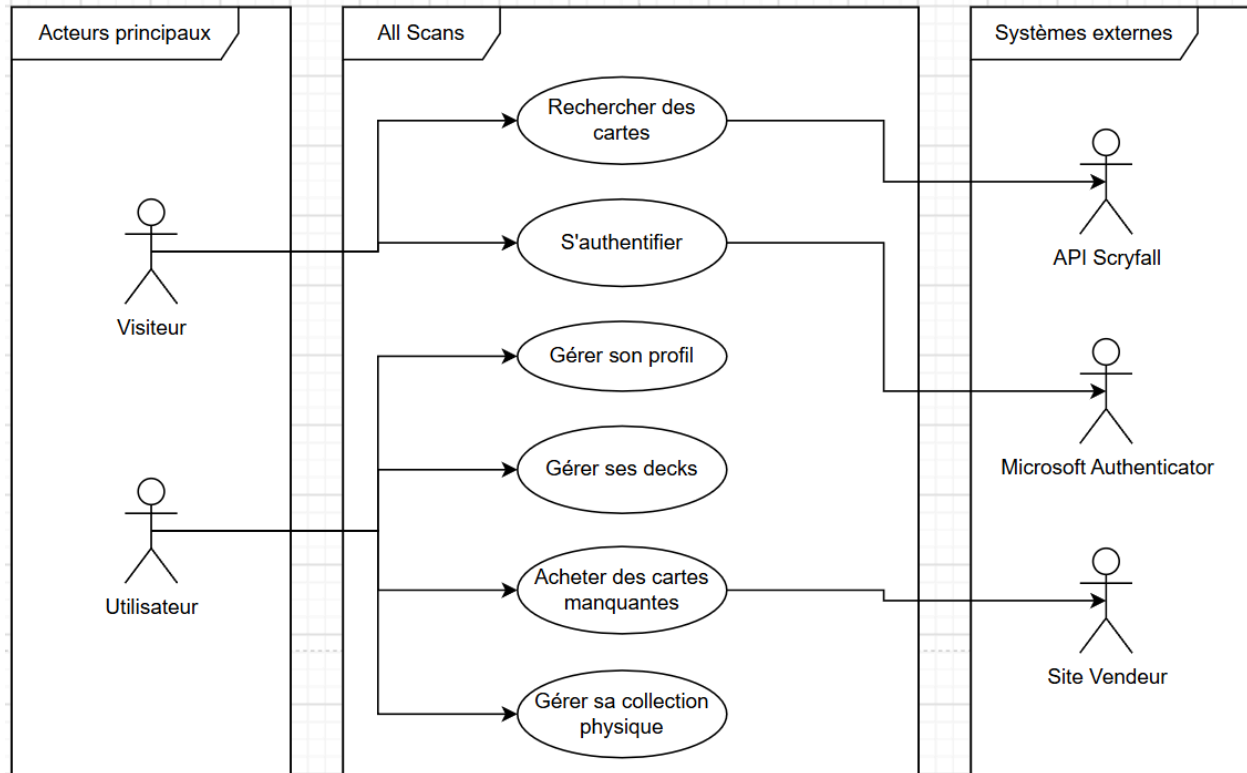
La structuration fonctionnelle de l'application s'articule autour des interactions entre les utilisateurs finaux (acteurs principaux) et le système "All Scans", ainsi qu'avec plusieurs systèmes tiers (acteurs secondaires) indispensables à l'exécution de processus spécifiques.

Les Acteurs Principaux Ce sont les entités humaines qui initient les interactions avec l'application :

-
- **Le Visiteur (Non authentifié)** : Son périmètre d'action est restreint. Il peut utiliser le moteur de recherche pour interroger la base de données de cartes de manière anonyme (*Rechercher des cartes*) et interagir avec le module de sécurité pour se connecter ou créer un compte (*S'authentifier*).
 - **L'Utilisateur (Authentifié)** : Il s'agit de l'acteur central du système. Une fois connecté, il débloque l'accès aux fonctionnalités métiers critiques de l'application : *Gérer son profil*, *Gérer sa collection physique* (ajout, modification et suivi de l'inventaire) et *Gérer ses decks* (construction, allocation des cartes et analyse statistique). Il peut également interagir avec le module permettant d'*Acheter des cartes manquantes*.

Les Systèmes Externes (Acteurs Secondaires) Ce sont les services logiciels tiers avec lesquels le système "All Scans" communique pour accomplir certains cas d'utilisation :

- **API Scryfall** : Fournisseur externe de la base de données mondiale de référence. Ce système est sollicité par l'application pour restituer les données exhaustives lorsqu'un utilisateur déclenche le cas d'usage *Rechercher des cartes*.
- **Microsoft Authenticator** : Application de sécurité tierce. Elle intervient de manière asynchrone lors du cas d'usage *S'authentifier* pour valider l'identité de l'utilisateur via la saisie d'un jeton temporaire (MFA / TOTP).
- **Site Vendeur** : Plateforme marchande externe vers laquelle l'application redirige l'utilisateur lorsqu'il sélectionne l'action d'*Acheter des cartes manquantes* pour compléter ses decks.



VI. Architecture logicielle

1. Architecture globale de l'application

Le projet repose sur une architecture moderne de type client-serveur, séparée en trois tiers distincts afin de garantir l'évolutivité et la maintenabilité du code :

- **Frontend (Présentation)** : Une Single Page Application (SPA) développée en React, gérant l'état local des interfaces et les interactions utilisateur de manière asynchrone.
- **Backend (Logique Métier)** : Une API RESTful développée en Python avec le framework FastAPI. Ce choix se justifie par ses hautes performances, sa gestion native de l'asynchronisme (essentiel pour les requêtes vers l'API externe Scryfall) et sa capacité à générer une documentation Swagger automatiquement. L'interfaçage avec la base de données est géré directement par le driver PyMongo.
- **Base de données (Persistance)** : Un système de gestion de base de données (SGBD) NoSQL orienté document, MongoDB.

2. Modélisation : Du Conceptuel au Physique

Le choix d'une architecture NoSQL orientée document (MongoDB) a été une décision d'ingénierie initiale et directe, motivée par deux contraintes techniques incontournables du projet :

1. **La mutation des données externes** : L'API Scryfall intègre régulièrement de nouvelles mécaniques de jeu, générant l'ajout imprévisible de nouveaux champs de données. Une base SQL stricte nécessiterait des migrations de schémas (ALTER TABLE) incessantes et risquées. Le format BSON de MongoDB offre la flexibilité indispensable pour ingérer ces mutations sans interruption de service.
2. **L'optimisation des lectures (Read-Heavy)** : L'affichage d'un deck complet avec ses statistiques dans un modèle relationnel exigerait de multiples requêtes de jointures coûteuses en performances. L'approche NoSQL permet le principe de "l'embarquement" (Embedding), regroupant les données liées en un document unique pour une restitution immédiate au client.

Cependant, afin de formaliser rigoureusement les règles de gestion métier complexes et de s'aligner sur les standards d'ingénierie logicielle attendus, la conception a d'abord fait l'objet d'une abstraction conceptuelle. Le **Modèle Logique de Données (MLD)** (*voir annexe : Schéma relationnel All Scans*) a été dressé pour cartographier les entités (utilisateurs, cartes, arborescence des dossiers, règles de tags) et identifier leurs cardinalités exactes, comme s'il s'agissait d'un système relationnel.

Le passage au **Modèle Physique de Données** (*voir annexe : Schéma non relationnel All Scans*) illustre l'adaptation de ce concept métier strict aux paradigmes d'optimisation du NoSQL.

3. Structure des Documents et Stratégies d'Optimisation

L'architecture physique se divise en plusieurs collections, illustrant des stratégies de modélisation avancées (Dénormalisation et Polymorphisme) :

A. Séparation Dictionnaire / Inventaire (Cards et UserCards) Plutôt que d'interroger Scryfall en temps réel pour chaque affichage, l'application maintient un dictionnaire local.

- La collection **Cards** agit comme un référentiel global, stockant l'exhaustivité des données d'une carte (ex: **oracle_id**, **legalities**, **prices**).
- La collection **UserCards** représente l'inventaire physique. Pour des raisons de performances (éviter les jointures à chaque affichage de l'inventaire), une stratégie de **dénormalisation** est appliquée : certaines données statiques de la carte (nom, image, rareté) sont dupliquées ici.
- *Règle d'allocation physique* : C'est dans ce document qu'est gérée la problématique centrale du projet via les champs **count** (possession totale) et **assigned_count** (nombre d'exemplaires physiquement verrouillés dans des decks construits).

```
// Exemple de structure d'un document UserCards
{
  "user_id": "697954da96845da5eb9a0d1d",
  "card_id": "57fb0fa7-0c5c-4a75-9461-c51403c30282",
  "count": 2,
  "assigned_count": 0,
  "is_foil": false,
  "name": "Foreboding Landscape",
  "image_normal": "https://cards.scryfall.io/..."
}
```

B. Polymorphisme et hiérarchie (Items) La table conceptuelle **ITEMS** a été traduite par une collection unique dans MongoDB, utilisant un champ discriminant **type** ("deck" ou "folder").

- L'arborescence est gérée par le champ **parent_id**, permettant une imbrication infinie (un deck dans un dossier, lui-même dans un autre dossier).
- Les tables de jointures conceptuelles ont été supprimées au profit de tableaux de références (tableaux **cards** et **sideboard**).

```
// Exemple de structure d'un document Item (Type: Deck)
{
  "user_id": "6a4908f8f6a907f1cad18b8d",
```

```

"type": "deck",
"nom": "Golbez",
"parent_id": null,
"format": "commander",
"is_constructed": false,
"commanders": ["95822b03-a49b-4a77-a257-fe0527088177"],
"cards": [
  "95822b03-a49b-4a77-a257-fe0527088177",
  "038710ca-c756-4e66-a9de-278e676c9f5b"
]
}

```

Exemple pour un document de type "folder" (dossier), illustrant la flexibilité du schéma où les champs spécifiques au deck (**format**, **is_constructed**, **commanders**) sont absents :

```

{
  "_id": "6a491538f6a907f1cad19cc7",
  "user_id": "6a4908f8f6a907f1cad18b8d",
  "type": "folder",
  "nom": "Erreur",
  "parent_id": null,
  "image": null,
  "cards": [],
  "sideboard": []
}

```

C. L'Embarquement de requêtes complexes (Tag_rules) L'implémentation du moteur de tags personnalisés démontre la puissance du modèle documentaire.

- Dans le modèle conceptuel, les conditions logiques ("contient le mot 'bat'", "contient le mot 'snake'") nécessitent une table fille **TAG_CONDITIONS**.

- Dans la collection `tag_rules`, ces conditions sont directement embarquées dans un tableau d'objets `conditions`, simplifiant drastiquement l'algorithmique de validation côté backend.

```
// Exemple de structure d'un document Tag_rules
{
  "user_id": "69d0fae014b7cd13bd129296",
  "tag_name": "nuisible",
  "logic": "OR",
  "color": "#8d852a",
  "conditions": [
    { "field": "type_line", "operator": "contains", "value": "bat" },
    { "field": "type_line", "operator": "contains", "value": "snake" }
  ]
}
```

4. Script d'initialisation de la base de données Bien que MongoDB utilise un schéma dynamique (NoSQL), la création d'index est indispensable pour garantir l'intégrité des données et les performances du moteur de recherche. Voici le script d'initialisation Python (via `pymongo`) exécuté pour configurer la base :

```
from pymongo import MongoClient, ASCENDING

client = MongoClient("mongodb://localhost:27017/")
db = client["All_scans"]

# Index composés pour accélérer les requêtes de recherche multicritères
db.user_cards.create_index([("user_id", ASCENDING), ("name",
ASCENDING)], name="user_id_1_name_1")
db.user_cards.create_index([("user_id", ASCENDING), ("colors",
ASCENDING)], name="user_id_1_colors_1")
db.user_cards.create_index([("user_id", ASCENDING), ("type_line",
ASCENDING)], name="user_id_1_type_line_1")
db.user_cards.create_index([("user_id", ASCENDING), ("cmc", ASCENDING)],
name="user_id_1_cmc_1")
```

```
# Index UNIQUE garantissant la non-duplication stricte d'une même carte physique
db.user_cards.create_index(
    [ ("user_id", ASCENDING), ("card_id", ASCENDING), ("is_foil",
ASCENDING) ],
    unique=True,
    name="user_card_foil_unique"
)
```

VII. Spécifications techniques

1. Organisation du code et conventions de nommage

L'architecture du code source respecte une séparation stricte des responsabilités (Separation of Concerns) afin de garantir la maintenabilité du projet à long terme.

- **Backend (FastAPI) :** Le code suit les standards PEP8 du langage Python. L'architecture est divisée en dossiers logiques : `routes/` (pour les contrôleurs de l'API), `models/` (pour les schémas Pydantic de validation), `utils/` (pour les fonctions d'aide comme le parseur textuel), et une centralisation de la connexion à la base de données. Les variables et fonctions utilisent la convention `snake_case`.
- **Frontend (React) :** L'application est structurée autour de composants fonctionnels modulaires isolés dans un répertoire `components/`. Le code JavaScript respecte les standards ES6+ et utilise le format `camelCase` pour les variables et fonctions, et le `PascalCase` pour les noms de fichiers et de composants (ex: `CardGrid.js`, `CardsList.js`).

2. Mise en conformité RGPD

L'application a été conçue en intégrant les principes du RGPD (Privacy by Design) pour assurer la sécurité et le respect de la vie privée des utilisateurs.

- **Minimisation de la collecte :** L'application ne collecte que les données strictement nécessaires à l'authentification et au fonctionnement du service (une adresse email,

un pseudonyme et un mot de passe). Aucune donnée personnelle superflue ou comportementale n'est requise.

- **Droit à l'effacement (Droit à l'oubli) :** L'utilisateur dispose d'un contrôle total sur ses informations. Une route de l'API est dédiée à la suppression de compte. Celle-ci implémente une suppression en cascade : elle efface le profil utilisateur, mais nettoie également de manière irréversible ses collections de cartes, ses decks et ses règles de tags personnalisées, garantissant qu'aucune donnée orpheline ne subsiste sur les serveurs.

3. Accessibilité

Bien que la nature de l'application (gestion de cartes à collectionner) soit intrinsèquement très visuelle, des normes d'accessibilité ont été appliquées lors du développement des interfaces pour se conformer aux bonnes pratiques du RGAA.

- **Alternatives textuelles :** Chaque image générée dynamiquement (notamment dans la grille de cartes ou les vues détaillées) intègre systématiquement un attribut HTML `alt` contenant le nom exact de la carte. Cela permet aux lecteurs d'écran de restituer l'information aux utilisateurs malvoyants, ou de pallier un défaut de chargement du réseau.
- **Contraste visuel :** La charte graphique utilise des variables CSS définissant des couleurs avec un niveau de contraste adapté entre les textes et les arrière-plans, réduisant la fatigue visuelle et facilitant la navigation.

4. Gestion des erreurs et résilience

La communication entre le client et le serveur est standardisée pour éviter tout comportement inattendu de l'interface en cas de défaillance.

- **Côté Serveur :** Le backend intercepte les anomalies (données manquantes, conflits d'allocation) et renvoie des exceptions HTTP typées (400, 401, 404, 500) accompagnées d'un message d'erreur clair encapsulé en JSON.
- **Côté Client :** Le frontend capture ces statuts via l'API `fetch`. Au lieu de faire planter l'application, l'interface affiche des notifications contextuelles ou des messages

d'erreur explicites directement dans les formulaires, guidant ainsi l'utilisateur vers la résolution de son problème.

5. Intégration et conformité de l'API Scryfall

L'application reposant fondamentalement sur les données de l'API Scryfall, le développement a été mené dans le respect strict de leur documentation technique et de la *Fan Content Policy* de Wizards of the Coast.

- **Gestion du trafic et identification** : Toutes les communications avec l'API s'effectuent de manière sécurisée via HTTPS. Conformément aux exigences du service, un en-tête HTTP `User-Agent` spécifique identifiant clairement l'application est transmis lors de chaque requête. Pour respecter les limites de requêtes (rate limits) imposées par Scryfall, les appels massifs au réseau (comme l'importation de listes entières) sont gérés de manière asynchrone et temporisés côté serveur via un système de traitement par lots (batching).
- **Intégrité des œuvres et attribution** : L'application préserve l'intégrité visuelle et légale des cartes. Les images ne sont ni déformées, ni altérées, et les mentions de copyright ou noms d'artistes ne sont jamais masqués artificiellement. Dans les interfaces spécifiques nécessitant l'utilisation d'une illustration recadrée (`art_crop`), l'application garantit que l'image complète de la carte (incluant les crédits officiels de l'artiste) reste systématiquement accessible à l'utilisateur au sein de la même interface (notamment via la modale de détails `CardModal`).

VIII. Réalisations

Cette section détaille les choix d'implémentation et la structure du code qui soutiennent l'application, en mettant en évidence les aspects critiques du système.

1. Développement Backend

Le backend de l'application est développé en Python et s'appuie sur le framework FastAPI. Ce choix technologique permet de bénéficier d'une haute performance grâce à la gestion native de l'asynchronisme, tout en assurant une validation stricte des données.

Structuration des routes API et cohérence des données

L'architecture du code favorise la modularité en séparant les responsabilités par domaine via la classe `APIRouter` de FastAPI. La sécurité est garantie par l'injection de dépendances systématique (`Depends(get_current_user)`) sur les routes sensibles.

Un point fort de l'implémentation réside dans la gestion de l'intégrité et de la cohérence des données lors des opérations de modification. Le système de tags automatisés illustre cette robustesse : lorsqu'une règle de tag est modifiée ou supprimée, le backend déclenche des opérations en cascade pour nettoyer la collection de cartes de l'utilisateur.

```
@router.delete("/rules/{rule_id}")
async def delete_tag_rule(rule_id: str, user_id: str =
Depends(get_current_user)):
    """Supprime une règle ET retire ce tag de toutes les cartes."""
    if not ObjectId.is_valid(rule_id):
        raise HTTPException(status_code=400, detail="ID de règle
invalide.")

    rule = tag_rules_collection.find_one({"_id": ObjectId(rule_id),
"user_id": user_id})
    if not rule:
        raise HTTPException(status_code=404, detail="Règle
introuvable.")

    tag_name = rule.get("tag_name")
    tag_rules_collection.delete_one({"_id": ObjectId(rule_id)})
    if tag_name:
        user_cards_collection.update_many(
            {"user_id": user_id,
            {"$pull": {"tags": tag_name}}
        )
    return {"message": "Règle supprimée et tags nettoyés sur vos
cartes."}
```

Interfaçage avec MongoDB et opérations asynchrones complexes

La persistance des données exploitée via MongoDB tire parti d'opérateurs avancés (pipelines d'agrégation, `$pull`, `$addToSet`) pour croiser les données de référence mondiales et les données spécifiques aux utilisateurs.

Pour garantir des performances de lecture optimales et maintenir une stricte intégrité, une architecture d'indexation avancée a été déployée sur les collections :

- **Optimisation des performances (*Compound Indexes*)** : Des index composés ont été créés pour anticiper les requêtes de recherche les plus lourdes. En liant systématiquement l'identifiant de l'utilisateur aux attributs clés des cartes (par exemple `user_id_1_name_1`, `user_id_1_colors_1`, `user_id_1_cmc_1`), le moteur de base de données peut résoudre les filtres complexes sans avoir à parcourir l'intégralité des documents.
- **Intégrité des données (*Unique Index*)** : Pour éviter les erreurs de logique métier, un index unique (`user_card_foil_unique`) a été appliqué sur le triplet `user_id`, `card_id`, et `is_foil`. Cette contrainte au niveau de la base de données garantit qu'un utilisateur ne peut posséder qu'une seule et unique entrée pour une version spécifique d'une carte, les ajouts ultérieurs incrémentant simplement le champ quantité.

▼ user_id_1	REGULAR ⓘ	192.5 kB	0 (since Mon Jul 20 2026)		READY
user_id ↑					
▼ user_id_1_name_1	REGULAR ⓘ	823.3 kB	0 (since Mon Jul 20 2026)	COMPOUND ⓘ	READY
user_id ↑ name ↑					
▼ user_id_1_colors_1	REGULAR ⓘ	311.3 kB	0 (since Mon Jul 20 2026)	COMPOUND ⓘ	READY
user_id ↑ colors ↑					
▼ user_id_1_type_line_1	REGULAR ⓘ	376.8 kB	0 (since Mon Jul 20 2026)	COMPOUND ⓘ	READY
user_id ↑ type_line ↑					
▼ user_id_1_rarity_1	REGULAR ⓘ	245.8 kB	0 (since Mon Jul 20 2026)	COMPOUND ⓘ	READY
user_id ↑ rarity ↑					
▼ user_id_1_cmc_1	REGULAR ⓘ	307.2 kB	0 (since Mon Jul 20 2026)	COMPOUND ⓘ	READY
user_id ↑ cmc ↑					
▼ user_card_foil_unique	REGULAR ⓘ	1.3 MB	0 (since Mon Jul 20 2026)	UNIQUE ⓘ COMPOUND ⓘ	READY
user_id ↑ card_id ↑ is_foil ↑					

La puissance de MongoDB est également exploitée via des pipelines d'agrégation dynamiques. La route `/cards/search` illustre parfaitement cette capacité en fusionnant les données via l'opérateur `$lookup` et en exécutant plusieurs sous-pipelines en une seule requête via l'opérateur `$facet` pour gérer simultanément la pagination et les métadonnées.

L'application brille particulièrement dans sa gestion des flux de données massifs en combinant cette base de données avec l'asynchronisme de Python. L'importation de collections entières repose sur un traitement asynchrone non-bloquant. L'API reçoit la requête, initialise un suivi d'état dans un dictionnaire global (`import_progress`), puis délègue le traitement lourd à une tâche de fond via `asyncio.create_task`.

Cela permet au client de recevoir une réponse immédiate et d'interroger dynamiquement la route `/usercards/import/progress` pour afficher une barre de progression en temps réel. Durant ce traitement en arrière-plan, le backend optimise les appels réseau en regroupant les requêtes vers l'API externe Scryfall par lots de 75 cartes (batching).

```
@router.post("/usercards/import")
async def start_import(request: Request, user_id: str =
Depends(get_current_user)):
    try:
        data = await request.json()
        uid = str(user_id)
        import_progress[uid] = {"total": len(data), "processed": 0,
"imported": 0, "status": "starting"}
        asyncio.create_task(perform_import(data, uid))
        return {"message": "Import lance", "total": len(data)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Logique métier et flexibilité des formats

Le backend intègre des modules de logique métier spécialisés répondant aux contraintes du domaine :

- **Interopérabilité** : Le système permet l'exportation fluide de la collection vers de multiples standards de l'écosystème (CSV, JSON, texte brut, ou formats spécifiques comme XML/DEK pour Magic The Gathering Online). La réponse HTTP est forgée dynamiquement avec le bon type MIME (`media_type`) et les en-têtes de téléchargement adéquats.
- **Moteur d'importation robuste** : Le parseur d'importation décode les listes textuelles et gère les subtilités des cartes (versions premium "Foil", numéros de collection, éditions spécifiques) avant d'appliquer les règles de tags automatiques définies par l'utilisateur.

Pour gérer l'importation de listes de cartes provenant de diverses plateformes (Arena, MTGO, Moxfield), l'application utilise un système de parsing textuel basé sur les expressions régulières. La fonction `parse_mtg_line` traite chaque ligne pour en extraire la quantité, le nom, l'édition, le numéro de collection, ainsi que des spécificités comme le format "Foil" ou l'appartenance au "Sideboard".

```
import re

def parse_mtg_line(line: str):
    """
    Analyse une ligne de texte de decklist MTG et extrait les
    informations.

    Supporte les formats Arena, MTGO, Moxfield, TCGPlayer, etc.
    """
    line = line.strip()
    if not line:
        return None

    is_foil = False
    is_sideboard = False
    is_commander = False

    # 1. Detection Sideboard / Commander en debut de ligne
    if re.match(r"^SB:\s+", line, flags=re.IGNORECASE):
        is_sideboard = True
        line = re.sub(r"^SB:\s+", "", line, flags=re.IGNORECASE).strip()
    elif re.match(r"^CMDR:\s+", line, flags=re.IGNORECASE):
        is_commander = True
        line = re.sub(r"^CMDR:\s+", "", line,
flags=re.IGNORECASE).strip()

    # 2. Nettoyage des balises de zone en fin de ligne (ex:
"(Principal)", "(Sideboard)")
    line =
re.sub(r"\s*\((?:Principal|Mainboard|Sideboard|Reserve|CMDR|Commander)\)
$", "", line, flags=re.IGNORECASE).strip()
```

```

# 3. Detection Foil
if re.search(r"\*[FE]\*$", line, flags=re.IGNORECASE) or
re.search(r"\bFoil\b", line, flags=re.IGNORECASE):
    is_foil = True
    line = re.sub(r"\*[FE]\*$", "", line,
flags=re.IGNORECASE).strip()
    line = re.sub(r"\bFoil\b", "", line,
flags=re.IGNORECASE).strip()

# 4. Regex complete: "4x Lightning Bolt (LEA) 234" ou "1 Black Lotus
[M14]"
pattern_full =
r"^(\d+) [xX]? \s+ (.+?) \s+ [ \(\[ ] ([a-zA-Z0-9]{2,5}) [ \)\] ] (?: \s+ (\d+ [a-zA-Z]
*) )? $"
match_full = re.match(pattern_full, line)

if match_full:
    return {
        "qty": int(match_full.group(1)),
        "name": match_full.group(2).strip(),
        "set": match_full.group(3).lower(),
        "collector_number": match_full.group(4) if
match_full.group(4) else None,
        "is_foil": is_foil,
        "is_sideboard": is_sideboard,
        "is_commander": is_commander
    }

# 5. Regex simple: "4x Lightning Bolt" ou "4 Lightning Bolt"
pattern_simple = r"^(\d+) [xX]? \s+ (.+) $"
match_simple = re.match(pattern_simple, line)

if match_simple:
    return {
        "qty": int(match_simple.group(1)),
        "name": match_simple.group(2).strip(),

```

```

        "set": None,
        "collector_number": None,
        "is_foil": is_foil,
        "is_sideboard": is_sideboard,
        "is_commander": is_commander
    }

# 6. Cas final: Aucun chiffre (ex: "Black Lotus")
return {
    "qty": 1,
    "name": line,
    "set": None,
    "collector_number": None,
    "is_foil": is_foil,
    "is_sideboard": is_sideboard,
    "is_commander": is_commander
}

```

2. Développement Frontend

L'interface utilisateur est développée en React, en s'appuyant exclusivement sur des composants fonctionnels et l'écosystème des Hooks natifs. Ce choix favorise un code moderne, facilement testable et performant.

Architecture des composants

L'architecture du projet distingue clairement les composants de présentation (dits "stupides" ou "dumb") des composants conteneurs (dits "intelligents" ou "smart"), ce qui favorise grandement la réutilisabilité du code.

- **Composants de présentation** : Le fichier `CardGrid.js` est un parfait exemple de composant réutilisable. Sa seule responsabilité est de recevoir des données via ses propriétés (`props` telles que `cards`, `renderAction`, `onCardClick`) et de les afficher sous forme de grille. Il embarque une logique interne d'affichage conditionnel très utile, comme la fonction `getImageUrl` qui cherche l'image de la carte à travers plusieurs propriétés possibles (face normale, double face, bordures coupées) pour garantir un affichage optimal, ou afficher un encart "Pas d'image" le cas échéant.



Éowyn, Lady of Rohan

Éowyn, Lady of Rohan Éowyn, Lady of Rohan

The Lord of the Rings: Tales ... x1 The Lord of the Rings: Tales o...x1

- **Composants conteneurs** : À l'inverse, [CardsList.js](#) est un composant central complexe qui orchestre la logique d'une page entière. Il gère dynamiquement l'affichage selon des onglets (Collection vs Import/Export) et coordonne le rendu de multiples sous-composants, comme l'appel conditionnel de [CardModal](#), [CollectionManager](#), ou [TagsManager](#).



Gestion de l'état

L'application privilégie les Hooks natifs de React plutôt qu'une bibliothèque de gestion d'état globale externe (comme Redux), ce qui allège le poids du client web tout en restant performant.

- **États locaux et complexes** : Le fichier `CardsList.js` démontre une gestion avancée de l'état via de multiples instances de `useState`. Il conserve en mémoire l'intégralité des filtres saisis par l'utilisateur (texte, couleurs exactes ou approximatives, rareté, force/endurance, statuts de légalité par format).
- **Gestion des flux multi-étapes** : La gestion de l'état s'applique également aux processus de sécurité. Le composant `LoginPage.js` modifie son interface dynamiquement via une variable d'état `step`. Si l'API backend signale que l'authentification multifacteur (MFA) est requise, le composant stocke le jeton temporaire (`mfaToken`) et bascule de l'étape 1 (Email/Mot de passe) à l'étape 2 (Saisie du code Google Authenticator à 6 chiffres) sans recharger la page.

Appels à l'API FastAPI et Optimisations de performance

Le frontend communique avec le backend FastAPI via l'API native `fetch`, en s'assurant d'inclure le paramètre `credentials: "include"` pour maintenir les sessions de l'utilisateur à travers les cookies ou jetons sécurisés.

Pour garantir une expérience utilisateur fluide malgré la potentielle lourdeur de la base de données, plusieurs optimisations techniques majeures ont été implémentées dans `CardsList.js` :

- **Défilement infini (Infinite Scrolling)** : Au lieu d'une pagination classique, le frontend utilise l'API `IntersectionObserver` combinée aux hooks `useRef` et `useCallback`. Le composant attache un observateur (`lastCardElementRef`) au dernier élément de la grille générée ; dès que cet élément devient visible à l'écran, l'état de la page s'incrémente (`setPage(prevPage => prevPage + 1)`), ce qui déclenche automatiquement la récupération du lot de cartes suivant.
- **Annulation des requêtes obsolètes** : Pour éviter les comportements erratiques liés à l'asynchronisme du réseau, la fonction de recherche utilise un `AbortController`. Si l'utilisateur modifie un filtre alors qu'une requête précédente est toujours en cours, le frontend annule l'ancienne requête HTTP via `controller.signal` avant d'en lancer une nouvelle.
- **Debouncing** : Pour ne pas inonder le backend de requêtes API à chaque lettre tapée dans la barre de recherche, un délai de 300 millisecondes (`setTimeout`) est imposé dans le `useEffect` écoutant les filtres, retardant ainsi l'exécution de la recherche tant que l'utilisateur continue de taper.

Filtres de Collection

Nom

Couleurs

Exacte



Type(s)

Texte

Rareté

Mot(s) clé(s)

Coût Mana (CMC)

Tag(s) actif(s) [\(Gérer les règles\)](#)

Légalité

Format (Tous)

Légal

Force

=

Endu.

=

Trier par

Nom

^

Interface de filtrage dynamique déclenchant la requête asynchrone `fetchCards`.

```
// Extrait de CardsList.js illustrant l'annulation de requêtes et la
pagination
const fetchCards = async (pageNumber, isNewFilter = false) => {
  // Annulation de la requête précédente si elle est encore en cours
  if (abortControllerRef.current) abortControllerRef.current.abort();
  const controller = new AbortController();
```

```

abortControllerRef.current = controller;

setLoading(true);

try {
  const params = new URLSearchParams();
  //construction de la requête
  if (searchTerm) params.append("name", searchTerm);
  if (oracleText) params.append("oracle_text", oracleText);
  if (rarityFilter) params.append("rarity", rarityFilter);

  if (colorFilter) {
    params.append("colors", colorFilter);
    params.append("color_mode", colorMode);
  }

  if (typeFilters.length > 0) {
    const typeQuery = typeFilters.map(t => t.mode === "exclude" ?
`-${t.text}` : t.text).join(",");
    params.append("type_line", typeQuery);
  }

  if (tagFilters.length > 0) {
    const tagQuery = tagFilters.map(t => t.mode === "exclude" ?
`-${t.text}` : t.text).join(",");
    params.append("tags", tagQuery);
  }

  if (keywordsFilter) params.append("keywords", keywordsFilter);
  if (cmcFilter) params.append("cmc", cmcFilter);

  if (powerFilter) {
    params.append("power", powerFilter);
    params.append("power_op", powerOp);
  }

  if (toughnessFilter) {

```

```

        params.append("toughness", toughnessFilter);
        params.append("toughness_op", toughnessOp);
    }

    if (formatFilter) {
        params.append("format_legality", formatFilter);
        if (legalityStatus) params.append("is_legal", legalityStatus);
    }

    if (setFilter) params.append("set_code", setFilter);

    params.append("page", pageNumber);
    params.append("limit", 60);
    params.append("sort_by", sortBy);
    params.append("sort_dir", sortDir);

    const endpoint =
`${API_BASE_URL}/cards/search?${params.toString()}`;

    const res = await fetch(endpoint, {
        credentials: "include",
        signal: controller.signal
    });

    if (!res.ok) throw new Error("Erreur chargement");
    const data = await res.json();
    // Concaténation des résultats pour le défilement infini ou
remplacement complet
    setCards(prevCards => isNewFilter ? data.cards : [...prevCards,
...data.cards]);
    setHasMore(data.cards.length === 60);

} catch (err) {
    if (err.name === 'AbortError') return;
    console.error(err);
} finally {

```

```
if (abortControllerRef.current === controller) {  
    setLoading(false);  
    abortControllerRef.current = null;  
}  
}  
};
```

3. Déploiement et Conteneurisation

Pour garantir la fiabilité, la portabilité et la facilité de déploiement de l'application, l'ensemble de l'infrastructure repose sur une architecture conteneurisée avec Docker. Ce choix technique permet de s'assurer que l'application s'exécute de manière strictement identique, qu'elle se trouve sur la machine locale d'un développeur, sur un serveur de test ou en production.

Le principe de l'environnement isochrone

La conteneurisation permet de créer un environnement dit "isochrone" (ou iso-production). Le système d'exploitation de base, les versions des langages (Python pour le backend, Node.js pour le frontend), les bibliothèques et les variables d'environnement sont définis de manière déclarative (Infrastructure as Code). Les conteneurs isolent les processus de l'application de l'hôte physique, éliminant ainsi les conflits de dépendances et le classique problème du comportement inattendu lié à l'environnement d'exécution.

Image Docker dédiée au Backend

L'image Docker du backend est conçue pour optimiser les performances et minimiser l'empreinte de l'application.

- **Image de base allégée** : L'utilisation de `python:3.11-slim` garantit une image légère tout en assurant une stricte compatibilité avec l'environnement de développement.
- **Optimisation des processus** : Les variables d'environnement `PYTHONDONTWRITEBYTECODE=1` et `PYTHONUNBUFFERED=1` sont configurées pour

empêcher l'écriture de fichiers `.pyc` inutiles sur le disque et pour forcer l'affichage des journaux (logs) en temps réel, ce qui est crucial pour le monitoring sous Docker.

- **Gestion propre des dépendances** : Le `Dockerfile` installe d'abord les paquets système requis pour la compilation (`build-essential`), puis installe les paquets Python en utilisant l'option `--no-cache-dir`. Cette méthode évite de conserver les fichiers d'installation en cache, réduisant drastiquement le poids du conteneur final.

```
# Extrait du Dockerfile Backend illustrant l'optimisation de l'image
# Utilisation de Python 3.11 pour correspondre à ton environnement local
# et GitHub
FROM python:3.11-slim

# Configuration des variables d'environnement
# Empêche Python d'écrire des fichiers .pyc
ENV PYTHONDONTWRITEBYTECODE 1
# Force Python à afficher les logs en temps réel (utile pour Docker
logs)
ENV PYTHONUNBUFFERED 1

# Définition du dossier de travail
WORKDIR /app

# Installation des dépendances système nécessaires (gcc, etc.)
# Utile si certaines librairies Python doivent être compilées
RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copie du fichier de dépendances
COPY requirements.txt .

# Installation des dépendances Python
# On upgrade pip d'abord pour éviter les warnings
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt
```

Orchestration avec Docker Compose

Le fichier `docker-compose.yml` (version 3.8) coordonne le cycle de vie des différents conteneurs et organise leur communication. Il définit une topologie réseau privée en mode `bridge` (`app-network`), isolant le trafic de l'application du reste du système hôte.

- Base de données (`mongo-db`) : Ce service déploie une instance de MongoDB (version 4.4). Le point névralgique de ce service est la déclaration du volume nommé `mongo-data`. Ce volume externe garantit la persistance des données (comptes utilisateurs, collections de cartes, historique d'importation) indépendamment du cycle de vie éphémère du conteneur.
- API (`backend`) : Le conteneur backend est instruit pour se lancer après la base de données via la directive `depends_on`. Il tire parti de la résolution DNS interne de Docker pour se connecter à MongoDB en utilisant simplement le nom du service (`MONGO_URI=mongodb://mongo-db:27017`). La configuration inclut un mappage de volume local (`./backend:/app`) qui, couplé à la commande `uvicorn ... --reload` du `Dockerfile`, permet le "hot-reload" : les modifications de code sont appliquées instantanément sans nécessiter de reconstruction de l'image.
- Interface utilisateur (`frontend`) : Ce service suit la même logique de développement fluide. Il démarre après le backend et expose le port 3000. Le volume configuré protège le répertoire `/app/node_modules` de l'écrasement local, et la commande de lancement `npm install && npm start` s'assure que les dépendances JavaScript sont toujours à jour au démarrage de l'environnement. La variable `REACT_APP_API_URL` permet au frontend de connaître dynamiquement l'adresse de l'API à requêter.

```
# Extrait du docker-compose.yml illustrant la résolution DNS et les
dépendances
backend:
  build: ./backend
  container_name: all-scans-backend
  restart: always
  ports:
    - "8000:8000"
  environment:
    - MONGO_URI=mongodb://mongo-db:27017
    - MONGO_DB_NAME=All_scans
  depends_on:
    - mongo-db
```

```
volumes:
  - ./backend:/app
networks:
  - app-network
```

IX. Sécurité de l'application

La sécurité du projet "All Scans" est traitée à travers une approche de défense en profondeur, protégeant à la fois les accès utilisateurs et l'intégrité de la base de données. L'architecture fait des choix techniques forts pour s'éloigner des implémentations standards parfois vulnérables.

1. Authentification et Autorisation

Le choix des jetons opaques

Plutôt que d'utiliser des JSON Web Tokens (JWT) qui sont "stateless" (sans état) et complexes à révoquer avant leur date d'expiration, l'application utilise des jetons opaques. Un jeton opaque est une chaîne de caractères aléatoire et dénuée de sens intrinsèque, générée cryptographiquement via `secrets.token_hex(16)`.

Ce choix technique oblige le serveur à vérifier le jeton dans la base de données à chaque requête, mais offre un avantage sécuritaire majeur : la révocation est instantanée. Lors d'une déconnexion ou d'une compromission, la suppression du jeton en base invalide immédiatement l'accès, ce qui est impossible avec un JWT classique sans recourir à une liste de révocation complexe.

Sécurisation du transport via les Cookies

Pour éviter le vol de jetons via des failles XSS (Cross-Site Scripting), le jeton n'est pas stocké dans le `localStorage` du navigateur. Il est injecté dans un cookie HTTP avec l'attribut `httponly=True` (le rendant invisible pour JavaScript) et `samesite="lax"` (pour prévenir les attaques CSRF).

```
# routes/auth_routes.py (Extrait de la génération de session)
token = secrets.token_hex(16)

# Enregistrement du jeton en base de données
users_collection.update_one({"_id": user["_id"]}, {"$set":
{"session_token": token}})
```

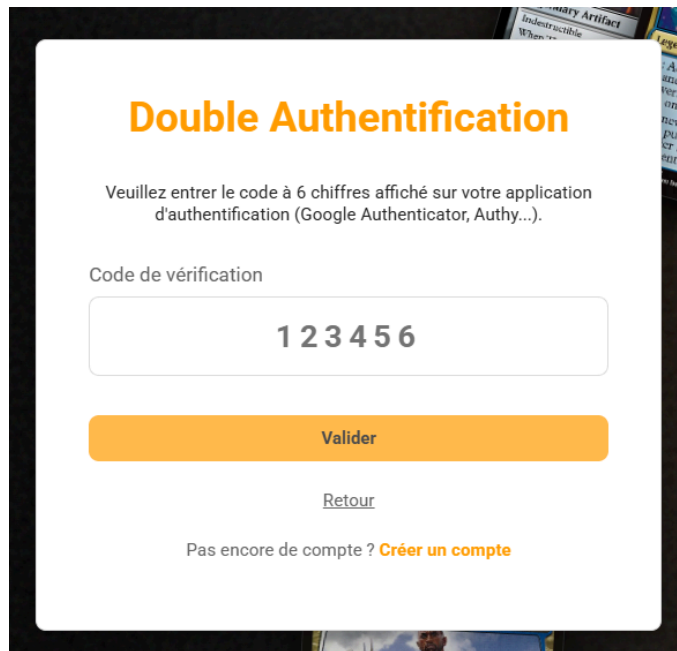
```
# Injection sécurisée dans le navigateur du client
response.set_cookie(
    key="session_token",
    value=token,
    httponly=True,
    samesite="lax",
    secure=False, # Passer à True en production sous HTTPS
)
```

Authentification Multifacteur

Pour contrer le vol d'identifiants, une couche MFA optionnelle est intégrée via la bibliothèque `pyotp`. Elle implémente le standard TOTP (Time-based One-Time Password), compatible avec Microsoft Authenticator et Google Authenticator. Si l'utilisateur a activé le MFA, l'API ne renvoie pas immédiatement le cookie de session final, mais crée un jeton temporaire stocké en mémoire dans `mfa_pending_sessions`.

```
# routes/auth_routes.py (Extrait de la vérification TOTP)
totp = pyotp.TOTP(user["mfa_secret"])
if not totp.verify(mfa_code):
    raise HTTPException(status_code=401, detail="Code MFA incorrect")

# Si le code est valide, on supprime la session temporaire et on génère
le vrai jeton
del mfa_pending_sessions[mfa_token]
```



Bascule dynamique du composant LoginPage vers l'état de validation MFA

2. Sécurisation des données en base

Le choix de l'algorithme Argon2

Les mots de passe ne sont jamais stockés en clair. Pour le hachage, le choix s'est porté sur **Argon2** (via la bibliothèque native `argon2-cffi`), le lauréat de la Password Hashing Competition (PHC). Contrairement à d'anciens algorithmes comme MD5 ou SHA-256 (qui sont trop rapides), ou même bcrypt, Argon2id est "memory-hard" (gourmand en mémoire). Cela signifie qu'il est spécifiquement conçu pour résister aux attaques par force brute menées par des processeurs graphiques (GPU) ou des circuits spécialisés (ASIC).

Implémentation et évolution

L'implémentation anticipe l'évolution de la puissance de calcul. La fonction `needs_rehash` permet de vérifier si un ancien mot de passe doit être recalculé avec de nouveaux paramètres de coût (temps/mémoire) lors de la prochaine connexion de l'utilisateur, assurant une sécurité pérenne.

```
passwords.py (Extrait de l'implémentation cryptographique)
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
```

```

import re

# Initialisation du hasher Argon2 natif
ph = PasswordHasher()

def hash_password(plain_password: str) -> str:
    """Hache le mot de passe avec Argon2id."""
    return ph.hash(plain_password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Vérifie si le mot de passe correspond au hash."""
    try:
        # La fonction verify de la librairie native renvoie True si
        # c'est bon,
        # mais lève une exception VerifyMismatchError si c'est faux.
        ph.verify(hashed_password, plain_password)
        return True
    except VerifyMismatchError:
        # Mot de passe incorrect
        return False
    except Exception as e:
        # Autre erreur (ex: hash corrompu)
        print(f"Erreur inattendue de vérification : {e}")
        return False

```

En amont du hachage, la robustesse des mots de passe est imposée par un validateur d'expressions régulières (`validate_password_strength`) exigeant 8 caractères, des majuscules, minuscules, chiffres et caractères spéciaux.

3. Protection contre les vulnérabilités courantes

Prévention des Injections NoSQL

L'utilisation de MongoDB rend les injections SQL traditionnelles impossibles. Cependant, MongoDB est sensible aux Injections NoSQL si les données entrantes (souvent du JSON) sont directement passées aux requêtes.

La défense est assurée sur deux fronts :

1. **Validation Pydantic** : FastAPI utilise Pydantic pour typer strictement les payloads entrants. Si un attaquant tente d'envoyer l'opérateur MongoDB `{"$ne": null}` à la place

d'une chaîne de caractères (pour contourner une vérification d'email), Pydantic rejettera la requête avec une erreur 400 (Bad Request) avant même qu'elle n'atteigne la logique métier.

2. **Cast BSON strict** : Toute recherche par identifiant de base de données (ID) passe obligatoirement par la classe `ObjectId()` de la bibliothèque `bson`.

```
# routes/auth_routes.py (Exemple de protection par ObjectId) # Si
user_id contient du code malveillant comme "; DROP...", # ObjectId()
lèvera une exception 'InvalidId' immédiate.

user = users_collection.find_one({"_id": ObjectId(user_id)})
```

Filtrage des requêtes externes

Pour protéger les utilisateurs contre l'exécution de requêtes silencieuses depuis des sites malveillants, l'API implémente un middleware CORS (Cross-Origin Resource Sharing). Celui-ci restreint la communication : seuls les domaines explicitement déclarés en liste blanche (ex: <http://localhost:3000> pour le frontend React) sont autorisés à lire les réponses du serveur et à transmettre des credentials (comme le cookie de session).

X. Plan de tests et jeu d'essai

La fiabilité de l'application "All Scans" repose sur une stratégie d'assurance qualité rigoureuse, matérialisée par une suite de tests automatisés (via le framework `pytest`). Cette stratégie couvre l'exactitude de la logique métier (tests unitaires), la cohérence des flux de données (tests d'intégration) et la résilience du système sous charge (tests de performance).

1. Présentation de la stratégie de test

L'architecture des tests suit une approche en pyramide, permettant d'isoler les responsabilités et de garantir la non-régression du code à tous les niveaux :

- **Tests Unitaires** : Ils valident le comportement isolé des fonctions pures et critiques. Par exemple, la classe `TestParsing` s'assure que les expressions régulières décodent correctement de multiples formats textuels, allant de lignes simples ("4 Lightning Bolt") à des syntaxes complexes incluant l'édition et le numéro de collection ("1 Sheoldred, the Apocalypse (DMU) 107a"). De même, la classe `TestExtraction` valide que la fonction `extract_card_fields` nettoie et standardise correctement les données brutes provenant de l'API Scryfall.

-
- **Tests d'Intégration et End-to-End (E2E)** : Ils simulent le parcours réel d'un utilisateur à travers l'API HTTP via le `TestClient` de FastAPI. La fonction `test_register_and_login_flow` valide ainsi le cycle complet de l'authentification : de la création du compte à la génération du cookie de session, jusqu'à la vérification de l'accès aux routes protégées et la déconnexion.
 - **Tests de Performance** : Ils préviennent les goulots d'étranglement de l'application en mesurant la consommation CPU locale et les temps de réponse de la base de données ou des API externes.

2. Tests unitaires et d'intégration

Robustesse de la logique métier (Test Unitaire)

La normalisation des données externes est une étape sensible. Le test unitaire `TestExtraction` s'assure que l'intégration de cartes depuis l'API Scryfall ne générera pas d'erreurs inattendues. Le test valide non seulement l'extraction des champs classiques (ID, nom, rareté, coût en mana), mais il simule également des cas limites : il vérifie que les champs `power` et `toughness` sont correctement gérés avec une valeur `None` pour les cartes non-créatures, et s'assure que l'absence de l'objet `image_uris` ne fait pas planter le système.

Validation des flux complexes (Test d'Intégration)

Le test `test_advanced_deck_flow` est un excellent exemple de validation métier de bout en bout. Ce script simule un flux utilisateur complet sur la gestion des decks :

1. **Injection** : Il insère d'abord un jeu de données mockées (3 cartes distinctes : un dragon, un sort et un terrain) directement dans la collection globale.
2. **Création et Ajouts** : Il crée un deck au format "commander", puis y ajoute plusieurs cartes via l'API, dont le terrain qui est ajouté deux fois de suite.
3. **Vérification de l'agrégation** : Le test valide que le backend ne crée pas de doublons mais incrémente bien la quantité du terrain à 2. Il vérifie également que les métadonnées (comme l'image ou le coût en mana) sont correctement récupérées depuis la base de données globale lors de l'appel GET.
4. **Suppression sécurisée** : Enfin, il teste la suppression partielle (retirer un seul terrain fait passer la quantité de 2 à 1) et totale (retirer le dragon l'efface complètement de la liste du deck).

Un test similaire, `test_card_collection_lifecycle`, applique cette même rigueur de validation pour le cycle CRUD (Create, Read, Update, Delete) de la collection principale de l'utilisateur, vérifiant que chaque modification de quantité est bien répercutée dans le moteur de recherche.

3. Tests de performance et de charge

Pour garantir une expérience utilisateur fluide, des scripts de tests de performance automatisés imposent des limites de temps de réponse strictes.

Scalabilité de la base de données (MongoDB)

L'application doit rester réactive même avec des collections d'utilisateurs volumineuses. Le script `test_search_speed_large_collection` met la base de données à l'épreuve en injectant artificiellement 2000 cartes générées dynamiquement dans la collection de l'utilisateur. Le test exécute ensuite une requête de recherche avec des filtres complexes (recherche de créatures de type "Goblin" avec la couleur rouge). Une assertion stricte garantit que ce processus de filtrage complet s'exécute en moins de 200 millisecondes (`MAX_ALLOWED_TIME_SECONDS = 0.200`).

```
# test_search_performance.py (Extrait de l'assertion de performance de
recherche)

end_search = time.perf_counter()
duration = end_search - start_search

# ASSERTION DE PERFORMANCE
assert duration < MAX_ALLOWED_TIME_SECONDS, \
    f"ALERTE : La recherche est trop lente ! ({duration:.4f}s >
{MAX_ALLOWED_TIME_SECONDS}s) "
```

Performances CPU et appels réseau

L'importation de listes de cartes sollicite à la fois le CPU (pour le parsing textuel) et le réseau (pour les requêtes Scryfall).

- Le test `test_parsing_performance` valide que le moteur d'expressions régulières est capable d'analyser 1000 lignes de texte en moins de 0.1 seconde.
- Le test `test_concurrent_batch_performance` utilise des appels asynchrones (`asyncio.gather`) pour envoyer 5 requêtes simultanées vers l'API Scryfall, s'assurant que l'ensemble du lot est traité et validé en moins de 2 secondes, confirmant ainsi que le serveur FastAPI gère efficacement la concurrence réseau sans bloquer le thread principal.

4. Jeu d'essai représentatif

Parseur d'importation textuel (`parse_mtg_line`)

Scénario testé	Entrée utilisateur (Chaîne brute)	Résultat Attendu	Résultat Obtenu	Statut
Ligne simple standard	"4 Lightning Bolt"	Qty: 4, Nom: "Lightning Bolt"	{qty: 4, name: "Lightning Bolt", set: None}	SUCCÈS
Ligne complète (Format Arena)	"1 Sheoldred, the Apocalypse (DMU) 107a"	Qty: 1, Nom: "Sheoldred, the Apocalypse", Edition: "dmu"	{qty: 1, name: "Sheoldred, the Apocalypse", set: "dmu"}	SUCCÈS
Carte brillante (Foil)	"1 Sol Ring *F*"	Nom: "Sol Ring", is_foil: True	{qty: 1, name: "Sol Ring", is_foil: True}	SUCCÈS
Carte en réserve	"SB: 2 Negate"	Nom: "Negate", is_sideboard: True	{qty: 2, name: "Negate", is_sideboard: True}	SUCCÈS

XI. Veille sécurité

La maintenance d'une application moderne et sécurisée nécessite une veille technologique constante. L'écosystème web évoluant rapidement, une méthodologie structurée a été mise en place pour anticiper les mises à jour majeures, optimiser les performances et se prémunir contre les nouvelles vulnérabilités.

1. Suivi de l'écosystème de développement

Pour garantir la pérennité de l'architecture technique, la veille s'appuie en priorité sur les sources officielles et communautaires des technologies utilisées :

- **FastAPI et Python** : Le suivi des évolutions de FastAPI se fait principalement via le dépôt GitHub officiel de son créateur (Sebastián Ramírez / Tiangolo), ce qui permet d'anticiper les changements liés à [Pydantic](#) et [Starlette](#) (sur lesquels FastAPI

repose). Les notes de version (Release Notes) de Python sont également consultées pour exploiter les nouvelles optimisations asynchrones ([asyncio](#)).

- **React (Frontend)** : La veille s'articule autour du blog officiel ([react.dev/blog](#)) pour suivre les nouveaux paradigmes (comme les Server Components ou l'évolution des Hooks). Les newsletters spécialisées (telles que *React Status*) et les conférences majeures (React Conf) permettent d'identifier les meilleures pratiques en matière de gestion d'état et d'optimisation du rendu.
- **MongoDB** : Le suivi des performances et des nouvelles capacités d'agrégation s'effectue via la documentation officielle et le blog d'ingénierie de MongoDB. Cela permet notamment d'optimiser les requêtes complexes utilisées dans le moteur de recherche de cartes.

2. Sécurité logicielle et prévention des failles (OWASP)

La sécurité de l'application est une préoccupation continue, intégrée dès la phase de conception (Security by Design). La veille sécuritaire s'organise autour des standards de l'industrie :

- **Référentiel OWASP (Open Worldwide Application Security Project)** : Le standard *OWASP Top 10* est utilisé comme grille de lecture pour auditer l'application. Les ressources de l'OWASP permettent de rester informé sur l'évolution des vecteurs d'attaque classiques (comme les failles XSS, les vulnérabilités de contrôle d'accès, ou les erreurs de configuration de sécurité) et d'adapter les middlewares en conséquence (gestion stricte des CORS, cookies HttpOnly).
- **Suivi des vulnérabilités (CVE)** : La veille implique la surveillance des bases de données de vulnérabilités (comme le NVD - *National Vulnerability Database*) pour détecter toute faille critique qui pourrait affecter les dépendances du projet (par exemple, une faille dans la bibliothèque [pyotp](#) ou [argon2-cffi](#)).
- **Outils d'analyse et d'alerte** : La plateforme GitHub est exploitée non seulement pour l'hébergement du code, mais aussi comme outil de veille active grâce aux *Security Advisories*.

Conclusion et axes d'amélioration

1. Bilan personnel sur le travail en autonomie

Le développement du projet full-stack "All Scans" a constitué un défi technique exigeant et particulièrement formateur. Concevoir et déployer une application de bout en bout en totale autonomie nécessite une forte polyvalence et une grande rigueur. Il a fallu assumer l'ensemble des rôles du cycle de vie logiciel : de l'architecture de la base de données NoSQL à la conception d'une interface utilisateur fluide en React, en passant par le développement d'une API asynchrone robuste sous FastAPI et la configuration de l'infrastructure via Docker.

Ce travail solitaire m'a poussé à rechercher et à appliquer des solutions architecturales pérennes. L'optimisation des performances de la base de données illustre parfaitement cette démarche : la mise en place d'index composés (*compound indexes*) tels que `user_id_1_name_1`, `user_id_1_colors_1` ou encore `user_id_1_cmc_1` démontre une anticipation proactive des montées en charge lors des requêtes de filtrage complexes. De plus, la création d'une contrainte d'unicité via l'index `user_card_foil_unique` sur le triplet (`user_id`, `card_id`, `is_foil`) garantit de manière stricte l'intégrité de la collection en empêchant la création de doublons au niveau même du moteur de base de données.

La résolution de ces problématiques d'ingénierie logicielle — couplée à la gestion sécurisée des sessions, l'intégration d'API tierces et le développement de parseurs sur mesure — a considérablement renforcé mes compétences globales. Cette expérience concrète confirme ma capacité à concevoir, développer et livrer un produit final robuste, sécurisé et performant.

2. Perspectives d'évolution de l'application

Bien que l'application soit pleinement fonctionnelle et réponde au cahier des charges actuel, l'architecture a été pensée pour soutenir plusieurs évolutions majeures, tant sur le plan logiciel que matériel :

- **Fonctionnalités sociales et E-commerce (Boutiques partenaires) :**

-
- L'objectif est d'ouvrir la plateforme aux professionnels en permettant aux boutiques spécialisées d'afficher leur inventaire.
 - Cette évolution s'accompagnera de la création d'un tag spécifique protégé ("en vente") pour identifier facilement les stocks commerciaux.
 - Côté utilisateur, la modale de recherche globale inclura un lien direct redirigeant vers la liste des boutiques partenaires possédant la carte consultée, créant ainsi un pont entre la gestion de collection et l'achat local.
 - **Développement Hardware (Station de tri automatisée) :**
 - Le projet ne s'arrête pas à l'interface logicielle. L'idéation et la sélection des composants électroniques pour la station de tri physique étant déjà finalisées, la prochaine grande étape consistera à achever la conception 3D de la machine pour lancer la phase de prototypage matériel.
 - **Internationalisation (i18n) :**
 - Afin de rendre l'application accessible à une audience plus large, l'intégration d'un module de gestion des traductions est prévue. Cela permettra d'ajouter un paramètre de sélection de la langue pour basculer l'interface dynamiquement (notamment en français et en anglais).
 - **Scalabilité de l'infrastructure :**
 - L'architecture actuelle est solide pour la base d'utilisateurs initiale. Cependant, en prévision d'une augmentation significative du trafic et des connexions simultanées, le déploiement évoluera vers l'intégration d'outils plus performants pour la répartition de charge et la gestion de bases de données distribuées, assurant ainsi une haute disponibilité continue.

XII. Annexes

Schéma relationnel All Scans :

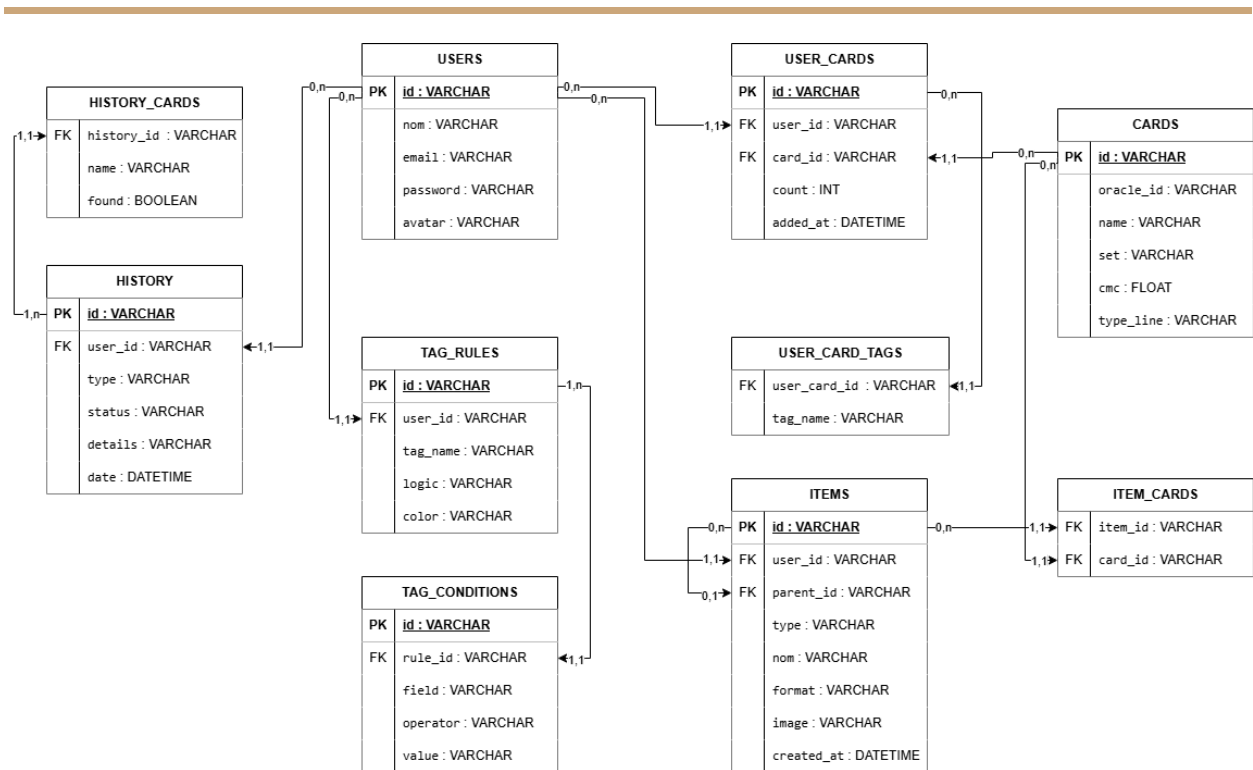
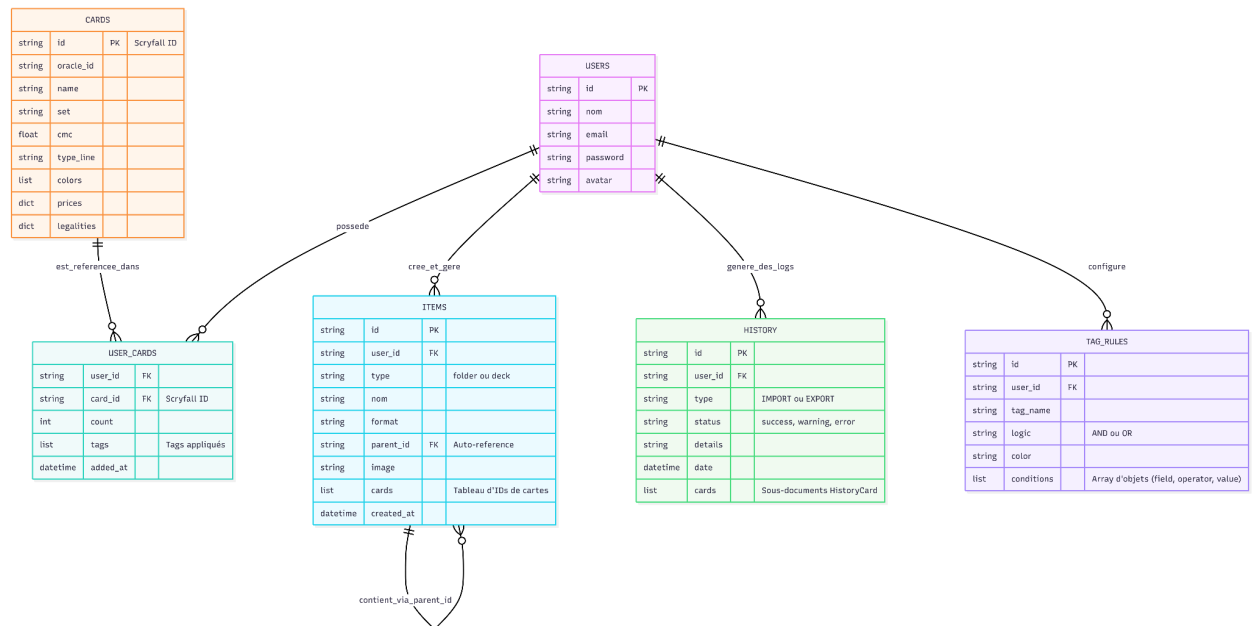
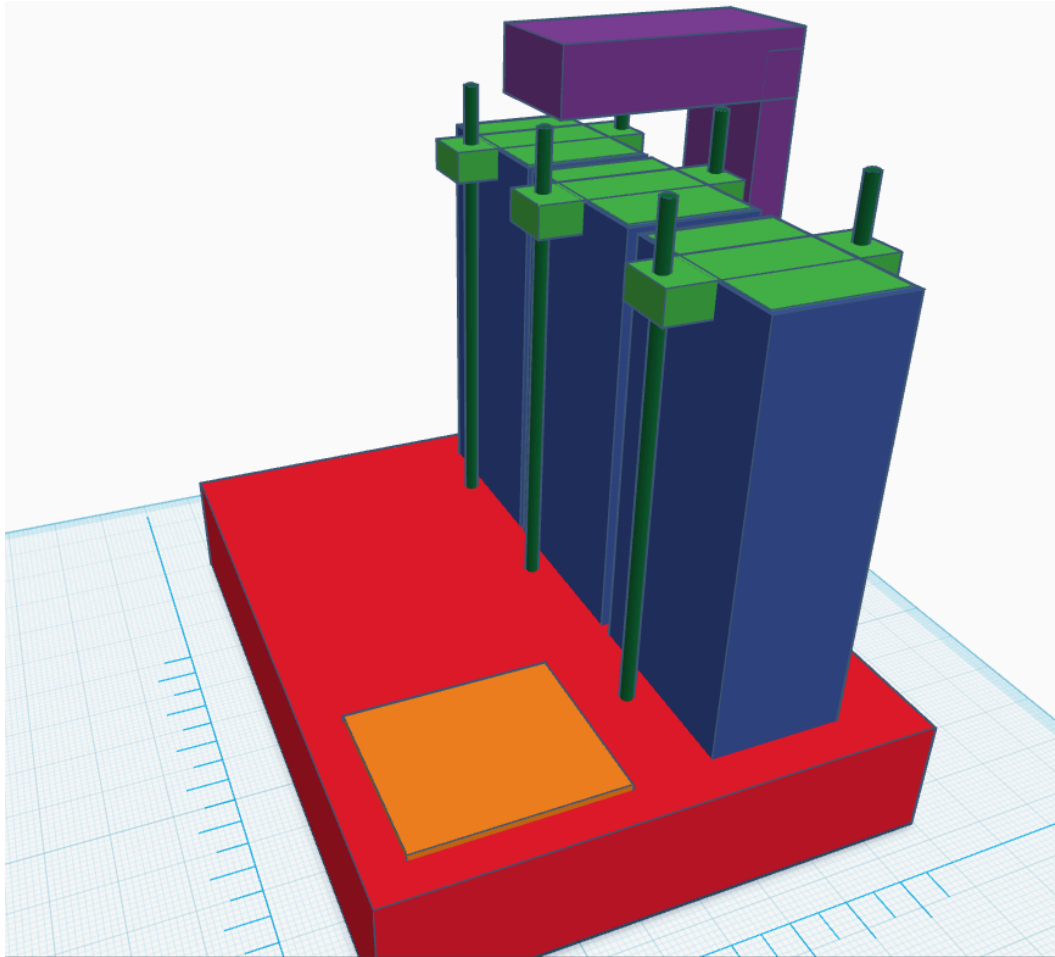


Schéma non relationnel All Scans :



Modélisation basique de la station de trie :



Liste des composants nécessaire :

Nom du composant	Quantité	Catégorie	Commentaire / Conseil technique
ESP32 (WROOM-32)	1	Cerveau Mécanique	Gère les moteurs, l'écran et le NFC.
ESP32-CAM (avec OV2640)	1	Cerveau Visuel	Se place en haut de la machine. Connecté en Wi-Fi à ton serveur. Prévois une nappe de caméra assez longue si besoin.
Antenne Wi-Fi (Optionnelle)	1	Réseau	Si la machine est loin de ta box, un ESP32-CAM avec prise pour antenne externe (IPEX) est un gros plus pour envoyer les images vite.
Alimentation 12V 10A	1	Énergie	Indispensable pour la puissance des 4 moteurs.
Convertisseur Buck	2	Énergie	Un réglé sur 5V pour l'ESP32 principal, un

(LM2596)			autre réglé sur 5V avec un bon condensateur dédié à l'ESP32-CAM (qui tire beaucoup de courant quand il flashe/émet en Wi-Fi).
Moteurs NEMA 17	3	Mécanique	Pour les ascenseurs (Racks).
Moteur NEMA 14 (ou Pancake)	1	Mécanique	Pour la roue de distribution des cartes.
Drivers TMC2209	4	Électronique	Indispensables pour le silence et la fluidité.
Tiges Filetées T8	3	Mécanique	Avec écrous anti-jeu (Anti-backlash) pour éviter que le rack redescende tout seul.
Tiges Lisses (8mm)	6	Mécanique	2 par rack pour le guidage linéaire (pour ne pas utiliser 6 tiges filetées !).
Roulements LM8UU	12	Mécanique	Pour glisser sur les tiges lisses.
Lecteurs NFC PN532	3	Capteurs	Module I2C, pour identifier les racks.
Multiplexeur I2C (TCA9548A)	1	Électronique	Pour brancher les 3 NFC et l'écran sans conflit d'adresse.
Tags NFC NTAG213	Lot	Consommable	À coller sous tes racks.
Écran OLED I2C	1	Interface	Pour afficher le statut de la machine et l'IP.
Encodeur Rotatif (KY-040)	1	Interface	Pour les menus.
Capteurs fin de course	3	Capteurs	Pour calibrer le zéro des ascenseurs au démarrage.
Bande LED blanche	1	Éclairage	À brancher sur l'ESP32 pour éclairer la carte sous la caméra de manière homogène (très important pour l'analyse d'image).